SIMATS ENGINEERING – DATA STRUCTURES (DSA0305)

ASSIGNMENT - I

NAME: ASWATHA V DATE: 26.10.2026

REG NO.: 192525390

1. What is Data Abstraction? Explain its Role in ADTs (5 Marks)

Definition

Data abstraction is the process of hiding the internal implementation details of data and showing only the essential operations to the user. It helps users understand and use data without knowing how it is implemented.

Role of Data Abstraction in ADTs

An Abstract Data Type (ADT) defines the data and the operations that can be performed on it without specifying how they are implemented.

1. Hides implementation: It hides the internal details of data storage and processing.
2. Provides simplicity: Users can perform operations without knowing the internal code.
3. Improves security: It prevents direct access to internal data.
4. Improves flexibility: The implementation can be changed without affecting the user.

Conclusion

Data abstraction makes ADTs simple, secure, flexible, and easy to maintain.

2. Design an Array ADT Using Arrays and Evaluate Performance (10 Marks)

Definition

An Array ADT is a collection of elements of the same data type stored in consecutive memory locations. Each element is accessed using an index.

Operations of Array ADT

1. Create: Creates an array with a specified capacity.
2. Insert: Adds an element at a specified position.
3. Delete: Removes an element from a specified position.
4. Search: Finds an element in the array.
5. Display: Prints all the elements.
6. Access: Retrieves an element using its index.

```
#include <stdio.h>
int main() {
    int a[100], n = 3, i, x = 20;

    a[0] = 10;
    a[1] = 20;
    a[2] = 30;

    printf("Array: ");
    for(i = 0; i < n; i++)
        printf("%d ", a[i]);

    for(i = 0; i < n; i++)
        if(a[i] == x)
            printf("\nFound at index %d", i);

    return 0;
}
```

Performance Evaluation

Operation	Time Complexity
Access	O(1)
Insertion at end	O(1)
Deletion	O(n)
Searching	O(n)
Display	O(n)

Advantages

1. Easy to implement.
2. Fast access using an index.
3. Stores multiple elements of the same type.

Disadvantages

1. Fixed size.
2. Insertion and deletion may require shifting elements.

Conclusion: Array ADT provides fast access to elements, but insertion and deletion can take more time.

Q3. Discuss Recursion in Compiler Design (5 Marks)

Definition

Recursion is a process in which a function calls itself to solve a problem.

Uses of Recursion in Compiler Design

1. Checking expressions: Helps the compiler understand expressions with brackets inside brackets.
2. Evaluating expressions: Helps process mathematical expressions step by step.
3. Checking grammar: Helps check the rules of a programming language.
4. Processing syntax trees: Helps the compiler process the different parts of a program.
5. Function calls: Helps a function call itself and manage those calls using a stack.

Advantages

- Makes nested expressions easier to process.
- Helps keep the program simple.

Disadvantage

Too many recursive calls can use too much memory and cause a stack overflow.

Conclusion

Recursion helps a compiler understand and process nested expressions, program rules, and function calls.